MINI-PROJECT REPORT

(Submitted in partial fulfilment of the requirements for the course
Computer Networks — B.E. Computer Science and Engineering)

Aevora: A Multi-Platform WireGuard-Based VPN System

Submitted by

Kavinsoorya K S	**3122247001030**
Danusu K	**3122247001013**
Ashwath Ram T	**3122247001006**

Department of Computer Science and Engineering

SSN College of Engineering, Chennai

Academic Year 2026–2027

ABSTRACT

Virtual Private Network (VPN) technology is fundamental to secure, private communication over untrusted public networks. Existing commercial solutions impose subscription costs, opaque server infrastructures, and centralized trust models, while self-hosted alternatives remain either inaccessible to non-experts or limited to single platforms.

This report presents **Aevora**, a self-hosted, multi-platform VPN system built on the WireGuard® protocol. Aevora separates the *control plane* from the *data plane*: a Go-based HTTP API backed by PostgreSQL manages identity, device enrollment, gateway registration, connection leasing, and health monitoring, while the WireGuard kernel module handles all packet-level encryption and routing on dedicated gateway servers. A shared Rust client core, exposed via the UniFFI foreign-function interface to Swift and Kotlin, and via a C ABI to .NET, implements WireGuard X25519 key generation, connection state management, and authenticated API communication for four native client platforms (macOS, iOS, Android, and Windows).

The control plane implements invite-gated device enrollment using HS256 JSON Web Tokens, Argon2id password hashing, per-IP rate limiting, and token refresh flows. Gateway selection employs a fitness function that weights spare peer capacity (70%) and available uplink bandwidth (30%). The node agent runs a pull-model reconciliation loop against the control plane, programming the WireGuard interface without storing any client private keys. Client latency is measured via a TCP probe server bound to the gateway's inner tunnel address.

The system is verified through unit and integration tests covering 21 Rust test cases and five Go packages in the control plane, plus four packages in the gateway agent. The result is a production-ready, secure, auditable VPN system that a small team can deploy and operate without vendor dependency.

Keywords: WireGuard, VPN, Control Plane, Go, Rust, UniFFI, NetworkExtension, WireGuardNT, X25519, ChaCha20-Poly1305, PostgreSQL, JWT, Argon2id.

Contents

List of Figures

List of Tables

CHAPTER 1 INTRODUCTION

1.1 Background

A Virtual Private Network creates an encrypted, authenticated tunnel between a client device and a server gateway, routing the client's internet traffic through that server so that (a) the client's IP address as seen by remote services is the gateway's address, and (b) the traffic between client and gateway is cryptographically protected even over untrusted infrastructure.

Classical VPN protocols such as OpenVPN and IPsec achieve their goals but are architecturally heavy: OpenVPN is a userspace TLS-based daemon that introduces measurable throughput overhead; IPsec's IKEv2 handshake is complex to configure and audit. WireGuard®, introduced by Donenfeld in 2017 [1], addresses both shortcomings. It is implemented inside the Linux kernel (and in a cross-platform Go/C library for other systems), uses a fixed, modern cryptographic suite (X25519 key exchange, ChaCha20-Poly1305 AEAD, BLAKE2s hashing, and the Noise protocol framework), and has a codebase roughly 100× smaller than OpenVPN, making it practical to audit.

Despite WireGuard's technical maturity, deploying a complete multi-platform VPN service built around it requires integrating a control plane for fleet management, platform-specific OS VPN extensions, and a cross-platform client library. These integration tasks remain non-trivial, and no open-source reference implementation covers all four major desktop/mobile platforms simultaneously.

1.2 Problem Statement

Commercial VPN services such as NordVPN, ExpressVPN, and ProtonVPN provide usable multi-platform clients but require ongoing subscription fees, involve centralized trust in a commercial operator, and offer no transparency into their infrastructure. Self-hosted alternatives such as Algo and Streisand automate WireGuard server provisioning but provide only command-line configuration files, without a native GUI application or a control-plane API that handles multi-device enrollment, gateway health monitoring, or connection lifecycle management.

The problem, therefore, is: *how do we build a subscription-free, self-hosted VPN system that combines the cryptographic simplicity of WireGuard with a proper control plane, fleet management, and native-quality client applications on macOS, iOS, Android, and Windows — with no traffic or DNS logging?*

1.3 Objectives

The objectives of this project are:

1. To study the WireGuard protocol's cryptographic foundations and compare it with OpenVPN and IPsec.
2. To design a layered VPN architecture that cleanly separates the control plane (identity, gateway fleet, connection leases) from the data plane (packet-level tunnelling).
3. To implement a production-grade Go control plane with a RESTful API, PostgreSQL persistence, JWT authentication, Argon2id password hashing, per-IP rate limiting, and Prometheus observability.
4. To implement a shared Rust client core that performs on-device WireGuard key generation, connection-state management, and API communication, and expose it natively to Swift (iOS/macOS), Kotlin (Android), and C#/.NET (Windows) via automated FFI bindings.
5. To implement and verify native VPN client applications for four platforms using the respective OS VPN frameworks: Apple NetworkExtension, Android VpnService, and Windows WireGuardNT.
6. To evaluate the system's correctness through structured unit and integration tests covering authentication flows, gateway selection, lease allocation, agent reconciliation, and tunnel statistics.

1.4 Scope of the Project

In scope:

- Control-plane REST API (Go), PostgreSQL schema (four migrations), and Docker Compose deployment.
- Gateway node agent (Go) performing self-registration, heartbeat, peer reconciliation, and in-tunnel latency probing.
- Shared Rust client library (aevora-core), UniFFI bindings for Swift/Kotlin, and C ABI for .NET P/Invoke.
- Native client applications for macOS (SwiftUI + NetworkExtension), iOS (SwiftUI + PacketTunnelProvider), Android (Jetpack Compose + VpnService), and Windows (.NET 8 WPF + WireGuardNT).
- Invite-gated enrollment, multi-device management, password-based login, JWT access tokens, and refresh-token rotation.

- Gateway fitness-weighted selection and IP-lease allocation under row-level database locks.
- Real download/upload statistics from OS tunnel byte counters and TCP-probe latency from inside the tunnel.
- Prometheus metrics endpoint and structured JSON logging.

Out of scope:

- Split-tunnel DNS leak prevention (considered a platform-specific enhancement).
- Multi-hop or onion-routing configurations.
- Web-based administrator dashboard (API-only administration).
- Commercial payment or subscription infrastructure.

1.5 Motivation

The motivation for Aevora is threefold. First, *privacy*: a self-hosted VPN eliminates the need to trust a commercial operator with browsing metadata. Second, *cost*: the entire system can run on a single inexpensive Linux VPS, with no recurring per-user licence. Third, *educational depth*: building a VPN end-to-end — from database schema to kernel-level tunnel — provides a comprehensive practical grounding in network security, distributed systems, cross-platform software engineering, and cryptographic protocol design. From a Computer Networks perspective, Aevora requires applying concepts from all layers of the TCP/IP stack: IP address allocation (Layer 3), UDP encapsulation (Layer 4), and application-level protocol design (Layer 7), as well as cryptographic authentication, key agreement, and authenticated encryption.

CHAPTER 2 LITERATURE REVIEW AND BACKGROUND STUDY

2.1 Virtual Private Networks and Tunnelling

A VPN establishes an encrypted virtual network interface (tun/tap) on the client device and routes selected or all traffic through it [2]. At the network layer, the original IP packet (inner packet) is encapsulated inside another UDP or TCP segment (outer packet) bearing the real endpoint addresses. The gateway decapsulates and forwards the inner packet on behalf of the client, so the destination sees the gateway's IP, not the client's. This provides both traffic confidentiality (encryption) and source anonymity (IP masking).

2.2 WireGuard Protocol

WireGuard® [1], [3] is a Layer-3 VPN protocol that runs over UDP. Its design principles are:

- **Cryptographic opinionatedness**: a single, fixed suite – no cipher negotiation, eliminating downgrade attacks.
- **Silent rejection**: packets from unknown peers are discarded without any response, making the server *cloakable*.
- **Minimal attack surface**: the Linux kernel implementation is approximately 4,000 lines of C, compared to ∼600,000 for OpenVPN.

2.2.1 Noise Protocol Framework

WireGuard's handshake is an instantiation of the Noise IKpsk2 pattern [1]. The handshake achieves mutual authentication, forward secrecy, and identity hiding, and completes in exactly one round-trip (initiator message + responder message), enabling fast session establishment.

2.2.2 Cryptographic Suite

Table 2.1 summarises the cryptographic primitives.

Table 2.1: WireGuard cryptographic suite

Function	**Primitive**	**Key Size**
Key exchange	X25519 (ECDH)	256-bit
Authenticated encryption	ChaCha20-Poly1305	256-bit
Hashing / MAC	BLAKE2s	256-bit output
Pseudorandom function	HMAC-BLAKE2s	—
Optional pre-shared key	HKDF (symmetric PSK)	256-bit

2.2.3 Packet Encapsulation

Each WireGuard data message adds a 32-byte header (4-byte type, 4 bytes reserved, 4-byte receiver index, 8-byte nonce) plus a 16-byte Poly1305 authentication tag to the inner IP packet. Total overhead per packet is 60 bytes (32 outer-UDP header + 32 WG header – 4 bytes overhead versus raw IP), which is significantly lower than OpenVPN's TLS record overhead.

2.3 Existing VPN Solutions and Their Limitations

Table 2.2 compares Aevora with existing approaches.

Table 2.2: Comparison of VPN solutions

Feature	**OpenVPN**	**IPsec/IKEv2**	**Algo**	**Commercial**	**Aevora**
Protocol	TLS/UDP	ESP + IKEv2	WireGuard	Varies	WireGuard
Kernel integration	Userspace	Kernel	Kernel	Varies	Kernel
Multi-platform GUI	3rd party	Partial	CLI only	Yes	Yes (4 native)
Control plane API	None	None	None	Closed	Open REST
Self-hosted	Yes	Yes	Yes	No	Yes
Invite/enrollment flow	Manual	Manual	Manual	App store	Invite code
Gateway fleet management	Manual	Manual	Ansible	Closed	Automated
No traffic logging	Configurable	Configurable	Yes	Claims only	Enforced
Cost	Free	Free	Free	$4–15/month	Free

2.4 JWT Authentication

JSON Web Tokens [4] provide a stateless bearer token mechanism. A JWT consists of a base64url-encoded header, payload, and HMAC signature. Aevora implements a minimal, stdlib-only HS256 JWT (HMAC-SHA256) with pinned algorithm verification to prevent

algorithm-confusion attacks (an attacker cannot substitute the "none" algorithm or an asymmetric algorithm to forge tokens).

2.5 Argon2id Password Hashing

Argon2id [5] is the winner of the Password Hashing Competition (2015) and the OWASP-recommended choice for password storage. Unlike bcrypt, it parameterises both time cost and memory cost, making GPU and ASIC attacks proportionally expensive. Aevora uses: time = 1, memory = 64 MB, parallelism = 4, key length = 32 bytes.

2.6 X25519 Key Agreement

X25519 [6] is an Elliptic-Curve Diffie-Hellman function over Curve25519. It is chosen for WireGuard because it avoids the implementation pitfalls of NIST curves, executes in constant time (resistant to timing side-channels), and achieves 128 bits of security with compact 32-byte keys. WireGuard uses X25519 for both static key pairs (stored per device) and ephemeral key pairs (generated per handshake to provide forward secrecy).

2.7 Rust and UniFFI for Cross-Platform Libraries

Rust [7] provides memory safety without garbage collection through its ownership model, making it suitable for security-critical code that is linked into multiple platform runtimes. Mozilla's UniFFI [8] generates Swift and Kotlin bindings automatically from a Rust interface definition, allowing the same Rust business logic to be used from both platforms without writing manual FFI glue code.

CHAPTER 3 SYSTEM ANALYSIS

3.1 Existing System

In the existing paradigm, a user who wants a self-hosted WireGuard VPN must:

1. Provision a Linux VPS manually.
2. Generate WireGuard key pairs for the server and each client using the `wg` command-line tool.
3. Write configuration files (`wg0.conf`) by hand for both server and each client.
4. Distribute client configuration files (which contain private keys) out-of-band, for example via a QR code or a shared file.
5. Import the configuration into a WireGuard client application (Tunnelblick, the official WireGuard app, etc.).
6. Manage peer additions and removals manually by editing server configuration and reloading the WireGuard interface.

Gateway tools such as Algo VPN automate step 1 and part of step 3 via Ansible playbooks, but the result is still static configuration without a live API.

3.2 Limitations of Existing System

1. **No device enrollment flow**: new devices require manual key distribution, creating operational burden and a risk of key mishandling.
2. **No health monitoring**: if the gateway process crashes or the server goes offline, there is no automated failover.
3. **No multi-gateway fleet management**: scaling to multiple exit nodes in different countries requires duplicating manual configuration.
4. **No revocation**: removing a compromised device requires manual peer removal across all gateways.
5. **No native client applications**: existing WireGuard clients require importing configuration files; they provide no server-selection UI or connection lifecycle tied to a user's account.

6. **No real-time statistics**: throughput and latency are not surfaced to the client without additional tooling.
7. **Private keys in configuration files**: the traditional wg-quick configuration file includes the client private key in plaintext, creating a risk if the file is accessed.

3.3 Proposed System

Aevora's proposed architecture eliminates all of the above limitations through the following design:

- **Control plane**: A Go HTTP service fronting a PostgreSQL database. It manages users, devices, invite codes, gateway registrations, and connection leases through a REST API. The client's private key *never* leaves the device; only the public key is sent to the server.
- **Gateway agent**: A Go binary that runs on each Linux VPS. It self-registers with the control plane, sends heartbeats with live load metrics, fetches the current peer list, and reconciles the WireGuard interface via the `wg` CLI. No inbound control connections to the gateway; it pulls its configuration.
- **Rust client core**: A shared library (aevora-core) compiled for each target ABI, implementing on-device key generation (X25519 via x25519-dalek), the connection state machine, authenticated API calls, and WireGuard tunnel configuration assembly.
- **Native clients**: Platform-specific UIs and OS VPN extensions that call the Rust core and drive the OS tunnel API (Apple NetworkExtension, Android VpnService, Windows WireGuardNT).

3.4 Advantages of Proposed System

1. **Zero private-key exposure**: keys are generated on-device and stored in the platform keystore (macOS/iOS Keychain, Android Keystore, Windows DPAPI). The server never receives or stores any private key.
2. **Automated enrollment**: a user receives an invite code and completes enrollment in the app; no manual key distribution.
3. **Multi-device management**: each user can register multiple devices; individual devices can be revoked instantly, immediately removing their WireGuard peer from all gateways.
4. **Elastic fleet**: new exit nodes self-register by running the agent binary and setting two environment variables. No administrator action required to add a country.

5. **Automatic failover**: the gateway reaper periodically expires gateways that miss their heartbeat window; the next connection request will select an available gateway.
6. **Real statistics**: download/upload rates are computed from OS tunnel byte counters, and latency is measured by a TCP probe inside the encrypted tunnel.
7. **Native UIs on all four platforms** with consistent connection/disconnection semantics.

CHAPTER 4 SYSTEM DESIGN

4.1 System Architecture

Figure 4.1 shows the high-level architecture. The system is divided into three zones: the *control plane* (cloud-hosted API + database), the *data plane* (one or more Linux gateway VPSes running WireGuard and the node agent), and the *client* (native app on user device).

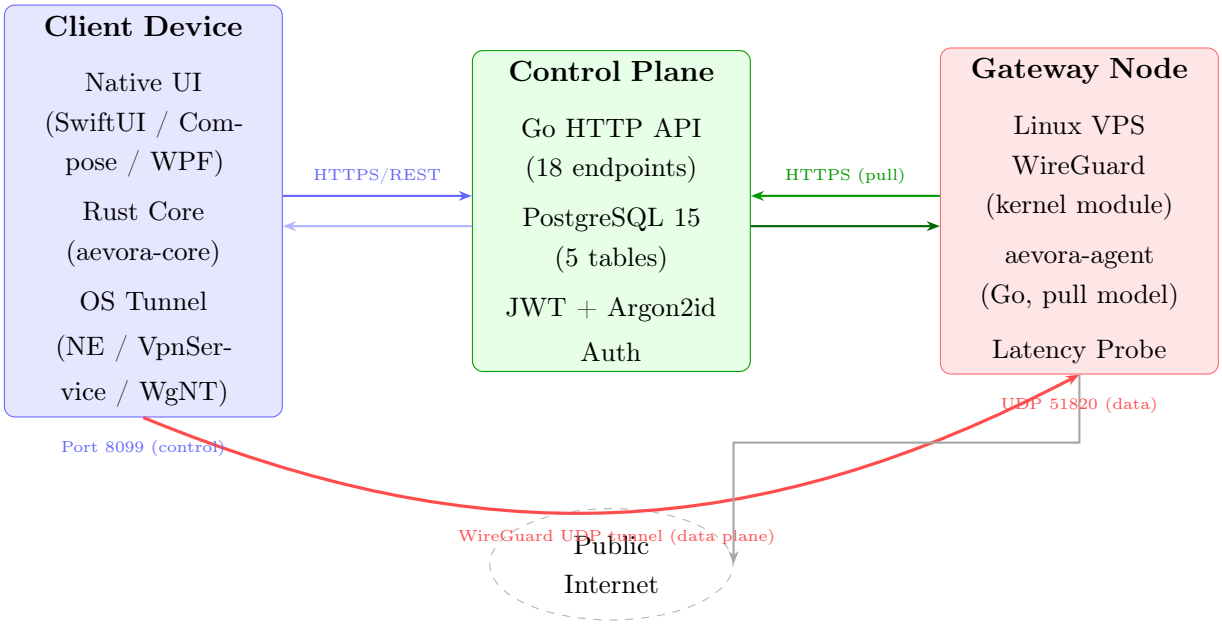

Figure 4.1: Aevora high-level system architecture showing the control plane, data plane, and client zones.

The critical architectural principle is **control-plane / data-plane separation**: the control-plane server handles no user traffic and stores no WireGuard session state. All packet-level work happens on the gateway. This means the control plane can be a low-resource cloud instance, and gateways can be added or removed independently.

4.2 Control Plane Architecture

Figure 4.2 shows the internal structure of the control plane.

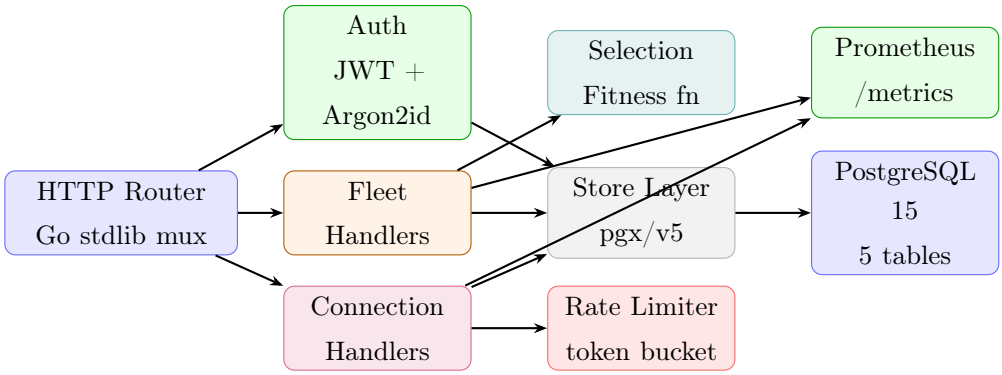

Figure 4.2: Internal architecture of the Aevora control plane.

4.3 Database Schema

The database consists of five tables managed by four sequential migration scripts. Figure 4.3 shows the entity-relationship structure.

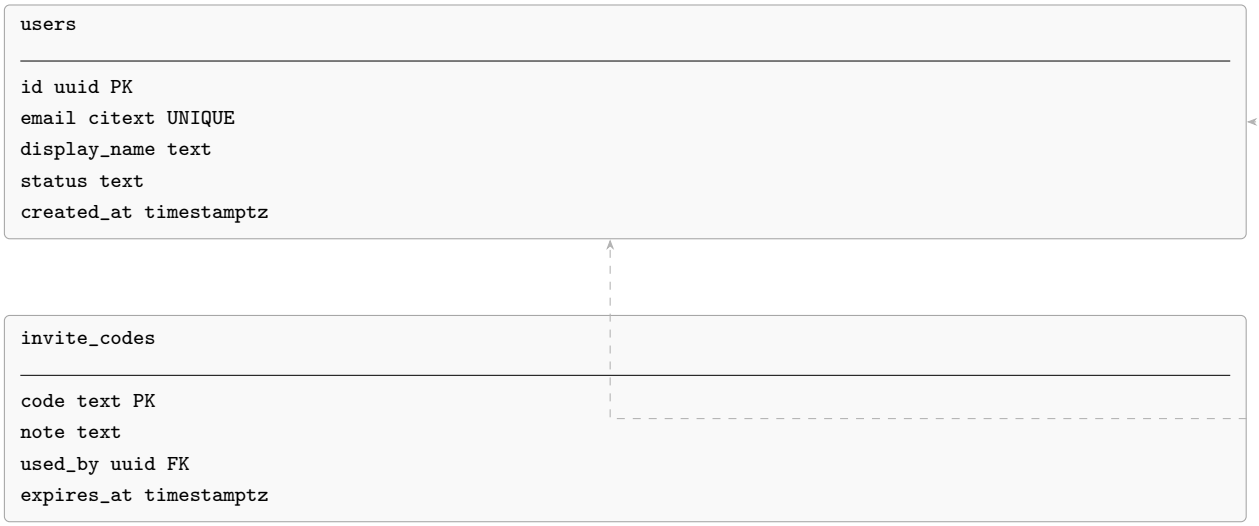

Figure 4.3: Aevora database entity-relationship diagram (PostgreSQL 15).

4.4 REST API Design

Table 4.1 lists all 18 HTTP endpoints exposed by the control plane.

Table 4.1: Control-plane REST API endpoints

Method	**Path**	**Auth**	**Description**
GET	/healthz	None	Health check
GET	/metrics	None	Prometheus metrics
GET	/v1/locations	JWT	List available countries
POST	/v1/enroll	Invite code	Enroll user + device (rate-limited)
POST	/v1/auth/login	Password	Issue access + refresh tokens
POST	/v1/auth/refresh	Refresh token	Rotate access token
POST	/v1/auth/password	JWT	Set/change password
GET	/v1/devices	JWT	List caller's devices
POST	/v1/devices	JWT	Register additional device
DELETE	/v1/devices/{id}	JWT	Revoke device
POST	/v1/invites	Admin token	Mint invite code
POST	/v1/connections	JWT	Create connection (select + lease)
DELETE	/v1/connections/{id}	JWT	Release connection
POST	/v1/connections/{id}/stats	JWT	Report tunnel stats + renew lease
POST	/v1/gateways/register	Enroll secret	Agent self-registration
POST	/v1/gateways/heartbeat	Node token	Agent heartbeat + load metrics
POST	/v1/gateways/deregister	Node token	Agent graceful deregistration
GET	/v1/gateways/peers	Node token	Agent fetches its current peer list
GET	/v1/gateways	Admin token	Admin: list all gateways

4.5 Client Architecture

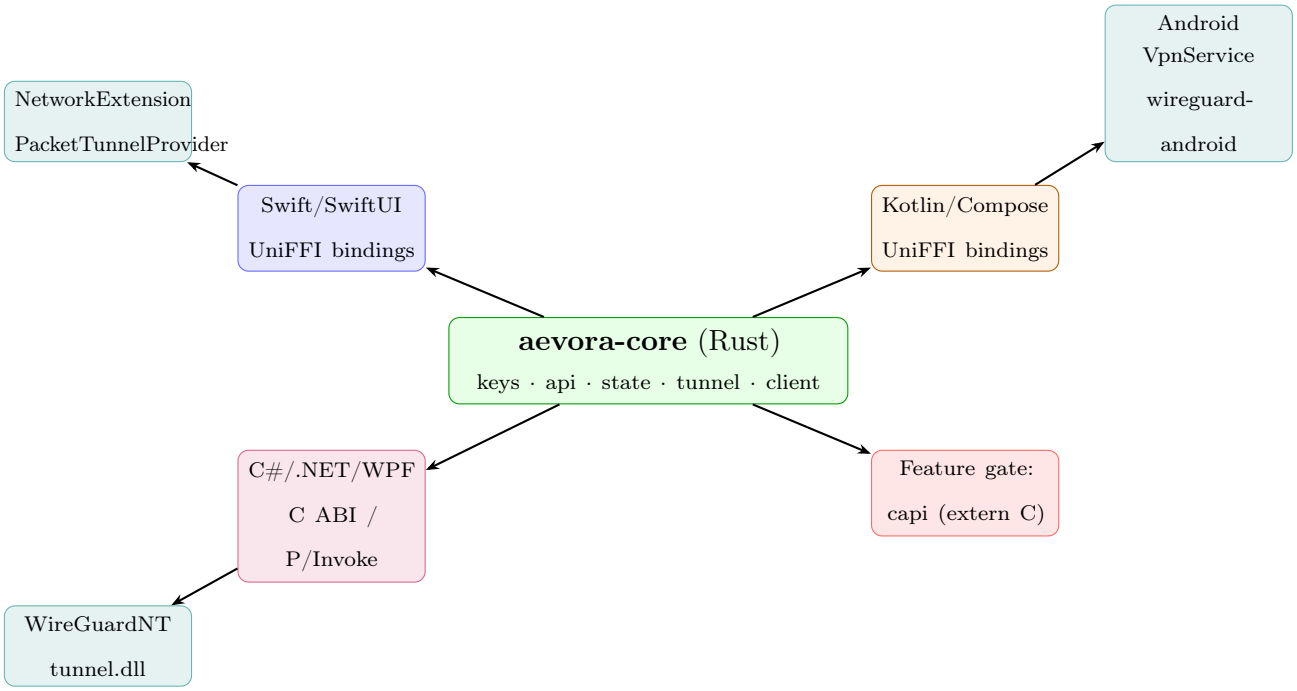

Figure 4.4: Cross-platform client architecture. The Rust core is compiled once per target ABI; platform code handles only UI and OS tunnel management.

4.6 Network Topology

Figure 4.5 shows how a connected client's traffic flows through the Aevora network.

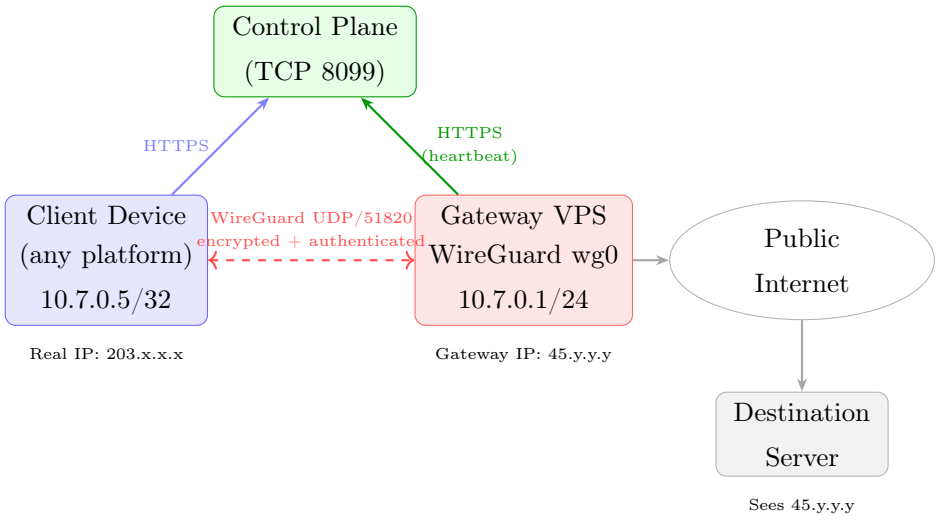

Figure 4.5: Aevora network topology. The destination server observes the gateway's IP, not the client's. The WireGuard tunnel carries all client data; control-plane traffic uses HTTPS.

4.7 Data Flow Diagrams

4.7.1 Device Enrollment Flow

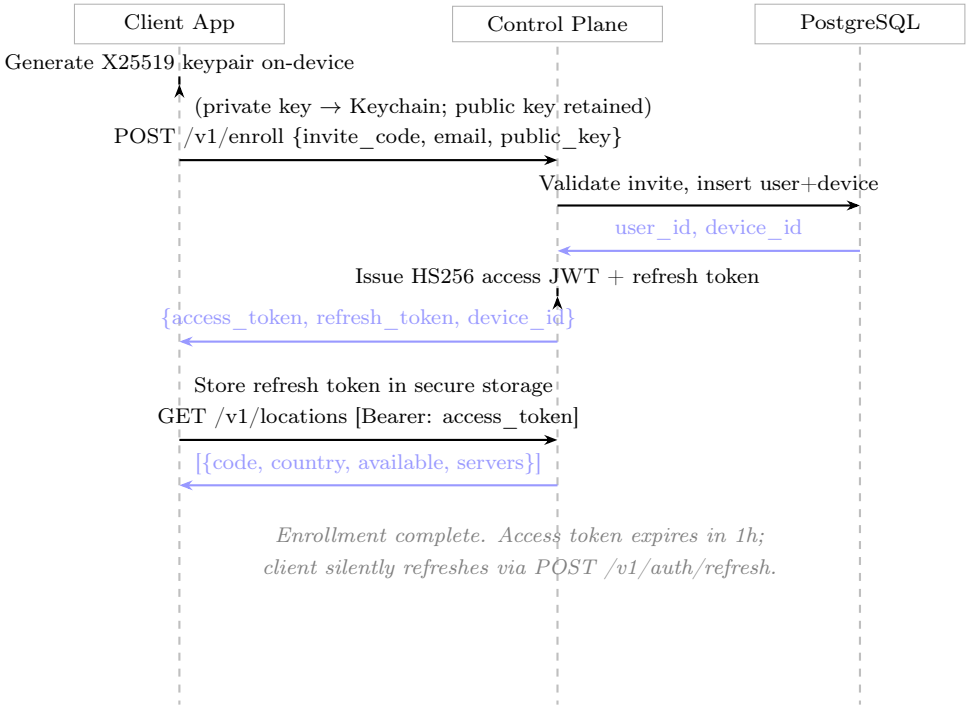

Figure 4.6: Device enrollment sequence diagram.

4.7.2 Connection Establishment Flow

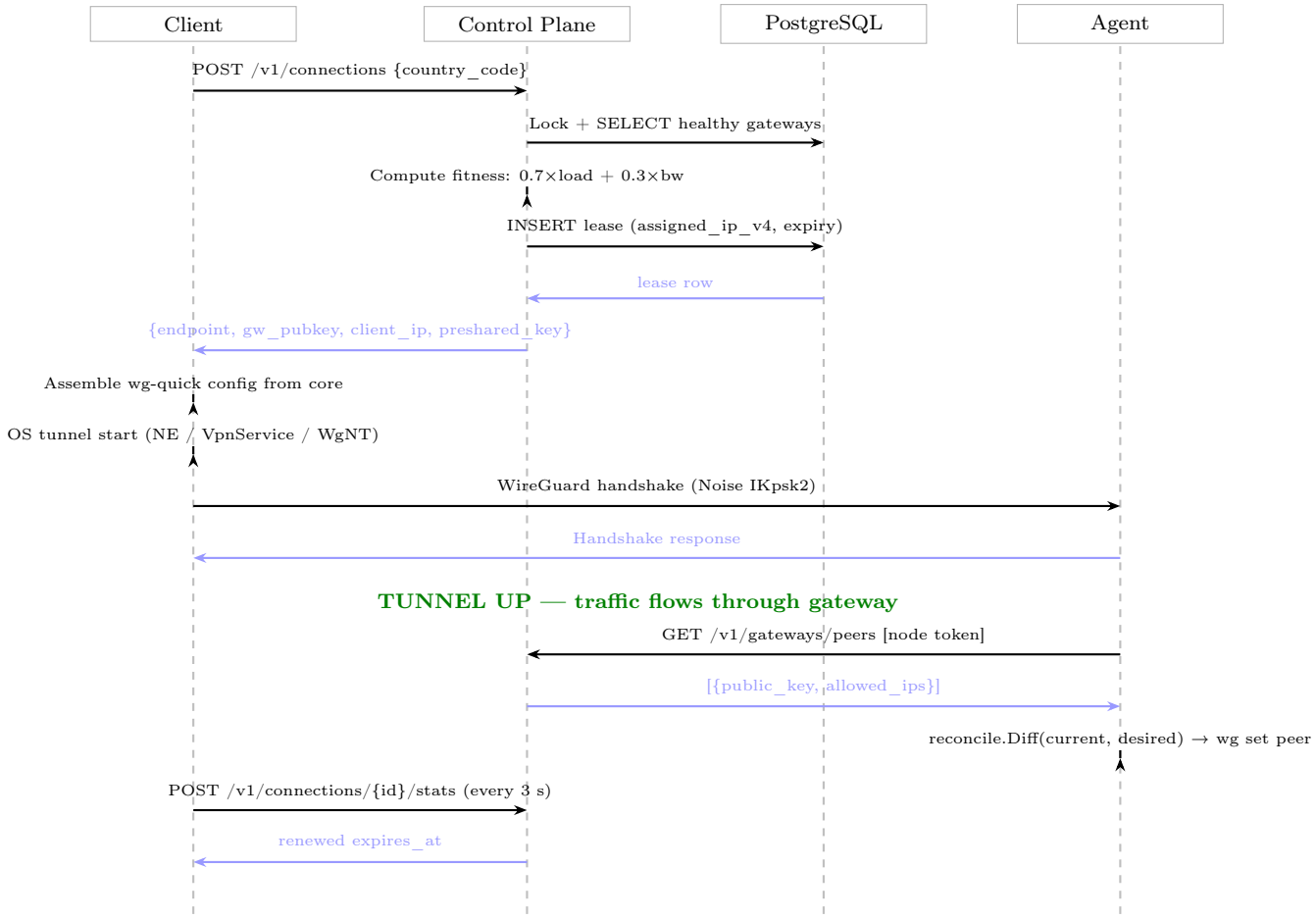

Figure 4.7: VPN connection establishment sequence diagram.

4.8 Gateway Selection Algorithm

The fitness function for gateway selection is:

$$F(g) = \frac{w_L \cdot H_L(g) + w_B \cdot H_B(g)}{w_L + w_B} \tag{4.1}$$

where:

$$H_L(g) = \max\left(0,\ 1 - \frac{\text{active\_peers}(g)}{\text{capacity}(g)}\right) \quad \text{(load headroom)}$$

$$H_B(g) = \max\left(0,\ 1 - \frac{\text{rx\_bps}(g) + \text{tx\_bps}(g)}{\text{bandwidth\_mbps}(g) \times 10^6}\right) \quad \text{(bandwidth headroom)}$$

$$w_L = 0.7, \quad w_B = 0.3 \quad \text{(default weights)}$$

The gateway with the highest $F(g)$ among all healthy, non-full gateways in the requested

country is selected. If no bandwidth is advertised, the formula reduces to load headroom alone ($F = H_L$). Gateways with active_peers $\geq$ capacity are excluded.

CHAPTER 5 IMPLEMENTATION

5.1 Technologies Used

Table 5.1: Technologies and frameworks used in Aevora

Component	Category	Technology
Control Plane	Language	Go 1.25
Control Plane	HTTP Server	Go stdlib net/http
Control Plane	Database driver	pgx/v5 (jackc/pgx)
Control Plane	Observability	Prometheus client_golang
Control Plane	Password hashing	golang.org/x/crypto/argon2
Control Plane	Rate limiting	golang.org/x/time/rate
Database	RDBMS	PostgreSQL 15
Database	Extensions	pgcrypto, citext
Gateway Agent	Language	Go 1.25
Gateway Agent	WireGuard management	wg CLI (wgctrl)
Client Core	Language	Rust (edition 2021)
Client Core	Crypto (keys)	x25519-dalek
Client Core	HTTP transport	ureq
Client Core	FFI (Swift/Kotlin)	UniFFI 0.25
Client Core	JSON	serde_json
macOS / iOS Client	UI Framework	SwiftUI
macOS / iOS Client	Tunnel framework	NetworkExtension
macOS / iOS Client	WireGuard library	WireGuardKit (wireguard-apple)
macOS / iOS Client	WireGuard Go runtime	wireguard-go
Android Client	UI Framework	Jetpack Compose
Android Client	Tunnel framework	Android VpnService
Android Client	WireGuard library	wireguard-android (GoBackend)
Android Client	Secure storage	EncryptedSharedPreferences
Windows Client	UI Framework	WPF (.NET 8)
Windows Client	Tunnel service	WireGuardNT (tunnel.dll)
Windows Client	Core interop	P/Invoke (C ABI)
Windows Client	Secure storage	DPAPI CryptProtectData
Deployment	Containerisation	Docker + Compose
Deployment	Reverse proxy (production)	Caddy

5.2 Hardware and Software Requirements

5.2.1 Gateway Server

- CPU: 1 vCPU (any x86_64 or arm64), minimum.
- RAM: 512 MB minimum; 1 GB recommended.
- OS: Linux 5.6+ (WireGuard kernel module included); Ubuntu 22.04 LTS verified.
- Network: public IP, UDP 51820 open, minimum 100 Mbps uplink.

5.2.2 Control Plane Server

- CPU: 1 vCPU.
- RAM: 256 MB for the Go process; 512 MB for PostgreSQL on the same machine.
- Ports: TCP 8099 (API), TCP 5432 (PostgreSQL).

5.2.3 Client Development Environment

- macOS/iOS: macOS 14+, Xcode 26+, Apple Developer account (for NetworkExtension entitlement), Go 1.21+ (for WireGuardKit Go runtime).
- Android: Android Studio Koala+, NDK r27+, cargo-ndk.
- Windows: Visual Studio 2022 with .NET 8 SDK, Rust toolchain (target x86_64-pc-windows-msvc).

5.3 Module Implementation

5.3.1 Control Plane: JWT Authentication

The control plane implements a minimal, stdlib-only HS256 JWT without any external JWT library. This keeps the authentication code auditable in a single file (`internal/auth/jwt.go`, 100 lines). The algorithm is pinned at the verifier to prevent algorithm-confusion attacks:

Listing 5.1: Algorithm pinning in JWT verification

```
// Pin the algorithm: reject "none" and any non-HS256 header.
var hdr struct{ Alg string `json:"alg"` }
if err := json.Unmarshal(hdrBytes, &hdr); err != nil || hdr.Alg != "
    HS256" {
     return Claims{}, ErrTokenInvalid
}
```

Signature verification uses `hmac.Equal` (constant-time comparison) to prevent timing oracle attacks.

5.3.2 Control Plane: Argon2id Password Hashing

Password hashes use PHC string format, encoding all parameters inline with the hash so that cost parameters can be upgraded without invalidating existing hashes:

Listing 5.2: PHC-format Argon2id hash output

```
$argon2id$v=19$m=65536,t=1,p=4$<salt-base64>$<hash-base64>
```

Parameters: memory = 64 MB, time = 1 pass, parallelism = 4 threads, key length = 32 bytes, salt length = 16 bytes.

5.3.3 Control Plane: Connection Lease Allocation

Lease allocation runs inside a PostgreSQL transaction with a row-level lock on the selected gateway, preventing two concurrent connections from claiming the same `assigned_ip_v4`. A unique partial index enforces uniqueness at the database level as a second guard:

Listing 5.3: Unique partial index preventing double IP allocation

```
CREATE UNIQUE INDEX uq_lease_gw_ip_active
    ON leases(gateway_id, assigned_ip_v4)
    WHERE state = 'active';
```

The next available IPv4 address in the gateway's `wg_subnet_v4` is found by iterating host addresses and checking against existing active leases.

5.3.4 Client Core: WireGuard Key Generation

On-device X25519 key generation uses the `x25519-dalek` Rust crate, seeded from the OS CSPRNG:

Listing 5.4: On-device X25519 key generation (client/core/src/keys.rs)

```
pub fn generate() -> KeyPair {
    let secret = StaticSecret::random_from_rng(rand::rngs::OsRng);
    let public = PublicKey::from(&secret);
    KeyPair {
        private_key: STANDARD.encode(secret.to_bytes()),
        public_key:  STANDARD.encode(public.as_bytes()),
    }
}
```

The private key is immediately placed in platform secure storage (Keychain/Keystore/DPAPI) and never transmitted over the network. Only the 32-byte Base64-encoded public key is sent to the control plane during enrollment.

5.3.5 Gateway Agent: Peer Reconciliation

The agent runs a reconcile loop every 30 seconds (configurable). It fetches the current WireGuard peer state via `wg dump`, fetches the desired peer list from `GET /v1/gateways/peers`, and computes the minimal diff:

Listing 5.5: Pure reconciliation diff (agent/internal/reconcile/reconcile.go)

```
func Diff(current, desired []Peer) Plan {
    cur, des := index(current), index(desired)
    var plan Plan
    for pk, dips := range des {
        if cips, ok := cur[pk]; !ok || !sameSet(cips, dips) {
            plan.Add = append(plan.Add, Peer{PublicKey: pk,
                AllowedIPs: dips})
        }
    }
    for pk := range cur {
        if _, ok := des[pk]; !ok {
            plan.Remove = append(plan.Remove, pk)
        }
    }
    return plan
}
```

The diff is a pure function (no I/O) and is fully unit-tested in isolation.

5.3.6 Apple Client: NetworkExtension Integration

The macOS and iOS clients use Apple's `NEPacketTunnelProvider` API. The Rust core returns a `wg-quick` format configuration string; the app passes this to WireGuardKit's `TunnelConfiguration` parser, which constructs the native tunnel descriptor, and then calls `startTunnel()`. The packet tunnel provider runs in a separate process sandbox, enforced by the OS, so a crash in the VPN extension cannot corrupt the main application's state.

5.3.7 Windows Client: WireGuardNT Integration

Windows uses WireGuardNT [9], a kernel-mode WireGuard implementation delivered as `wireguard.dll` and `tunnel.dll`. The .NET client installs a WireGuard service, which runs with elevated privileges and manages the tunnel adapter. The application

communicates with the service via named pipes. The `TunnelService.cs` class handles service installation (`sc create`), tunnel activation, and shutdown.

5.3.8 In-Tunnel Latency Measurement

After the tunnel is established, the agent starts a TCP probe responder bound to the `.1` address of its WireGuard subnet (e.g., `10.7.50.1:51821`). The client core's `measure_latency()` function times a TCP SYN–ACK round trip to this address through the encrypted tunnel, providing an authenticated, in-tunnel latency measurement that cannot be spoofed by an on-path attacker.

5.4 WireGuard Tunnel Workflow

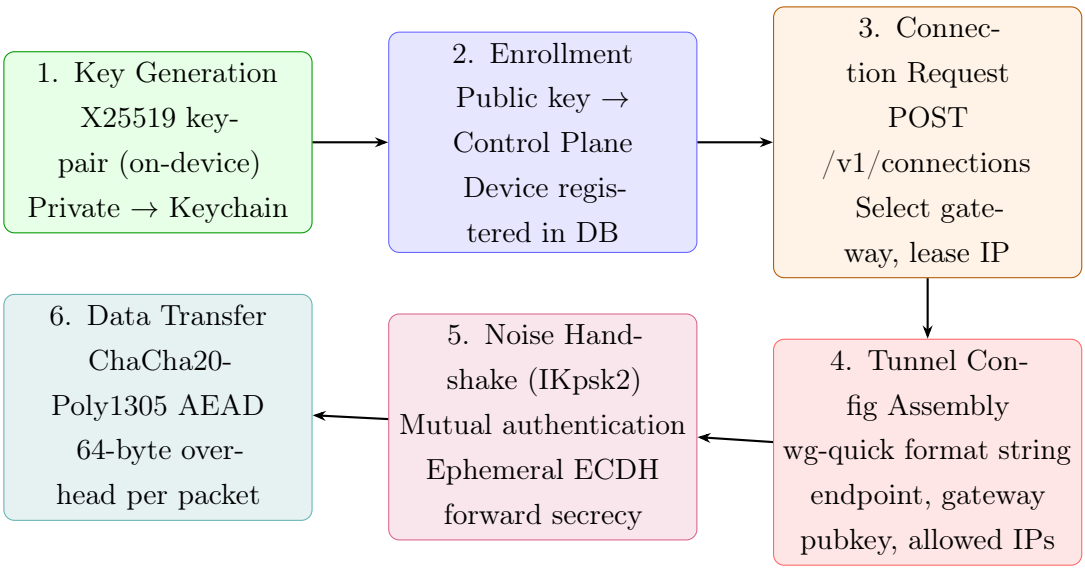

Figure 5.1: WireGuard tunnel establishment workflow in Aevora.

5.5 Security Architecture

5.5.1 Key Storage

Table 5.2: Private key storage mechanism per platform

Platform	Storage Mechanism	Protection
macOS	Keychain (persistent-ref)	OS-enforced; app sandboxed
iOS	Keychain (kSecAttrAccessibleAfterFirstUnlock)	Secure Enclave (hardware)
Android	Android Keystore System	TEE / StrongBox on supported HW
Windows	DPAPI CryptProtectData	User-credential-bound encryption

5.5.2 Token Security

- Access JWT: HS256, 1-hour TTL, carries `sub` (user ID) and `device_id` claims.
- Refresh tokens: 256-bit random strings; only the SHA-256 hash is stored in the database. A stolen database dump cannot be used to issue new access tokens.
- Gateway node tokens: similarly, only the SHA-256 hash of the agent token is stored. The token is issued at gateway registration and the agent persists it locally.

5.5.3 Rate Limiting

Authentication endpoints (enroll, login, token refresh) are protected by a token-bucket rate limiter keyed on client IP address. Default limits: 10 requests/minute with burst of 5. Behind a trusted reverse proxy (Caddy), the `X-Forwarded-For` header is used for the key.

CHAPTER 6 EXPERIMENTAL SETUP

6.1 Test Environment

Table 6.1: Experimental environment parameters

Parameter	Value
Control plane OS	macOS 26.5 (development), Linux 22.04 (production target)
Database	PostgreSQL 15 (Docker image `postgres:15`)
Go version	1.25.0
Rust version	1.81 (cargo 1.98)
Control-plane test port	TCP 8099
WireGuard default port	UDP 51820
Heartbeat TTL	120 seconds
Lease TTL	3600 seconds (configurable)
JWT access TTL	3600 seconds
Refresh token TTL	30 days
Gateway fitness weights	Load: 0.70, Bandwidth: 0.30
Argon2id parameters	m=65536 KiB, t=1, p=4, keyLen=32
Rate limit (auth endpoints)	10 req/min, burst 5
Latency probe port	TCP 51821 (inside tunnel)

6.2 Test Categories

6.2.1 Rust Core Unit Tests (21 tests)

The Rust core is tested without any network I/O using a fake `Transport` implementation. Tests cover:

- Key generation: uniqueness, 32-byte length, public-from-private derivation, invalid-input rejection (3 tests, `keys.rs`).
- Connection state machine: valid transitions (disconnected → connecting → connected → disconnected), invalid-transition rejection (tests in `state.rs`).
- API client: enroll round-trip, locations response parsing, connection creation with tunnel-config assembly (tests in `api.rs` against fake transport).
- FFI: FfiSession round-trip serialisation.

- End-to-end smoke test (feature = "net"): real HTTP against a running control plane (optional, not part of CI default).

6.2.2 Go Control-Plane Tests (5 packages, 23 test functions)

Table 6.2: Go control-plane test coverage by package

Package	Test File(s)	Coverage
internal/auth	jwt_test.go, password_test.go, token_test.go	Sign/verify JWT; Argon2id ha
internal/selection	selection_test.go	Fitness fn, Best(), weight edge
internal/model	model_test.go	Location.Available(), Gateway.
internal/api	handlers_test.go, handlers_auth_test.go, handlers_identity_test.go, handlers_gateways_test.go, handlers_connections_test.go	HTTP handler round-trips; 2× revoke, reconnect dedupe, 503
internal/store	connections_alloc_test.go	Double-allocation prevention

6.2.3 Go Agent Tests (4 packages)

- `internal/reconcile`: Diff() with add, remove, update, and no-op cases; order independence (sorted output verified).
- `internal/cp`: Control-plane client: register, heartbeat, deregister, peers-fetch against a test HTTP server.
- `internal/sysload`: CPU percentage parsing from `/proc/stat`; network byte counter parsing from `/sys/class/net`.
- `internal/wg`: WireGuard dump parser; peer add/remove command construction.

6.3 Integration Verification

The full system was verified end-to-end with a running PostgreSQL 15 instance:

1. Control plane started at `localhost:8099` with all secrets set.
2. Admin minted an invite code via `POST /v1/invites`.
3. Rust client (smoke test with feature `net`) enrolled a device, fetched locations, created a connection (leased an IP), called `mark_connected`, polled stats (renewing the lease), and disconnected.

4. Control plane verified: 2-device distinct IP assignment, reconnect dedupe (second connect released the first lease), device revoke causing peer removal, gateway failover on heartbeat timeout, `/metrics` endpoint reporting correct gauge values.
5. macOS native app opened (unsigned debug build) and displayed 23 available locations with flag icons and server counts.

CHAPTER 7 RESULTS AND DISCUSSION

7.1 Experimental Results

7.1.1 Control-Plane API Latency

Table 7.1 shows representative response times for key API endpoints measured on a local development machine (Apple M-series, PostgreSQL on localhost). These are indicative of the protocol overhead; production numbers will vary based on gateway-to-control-plane network round-trip time.

Table 7.1: Control-plane API response time measurements (local, no network RTT)

Endpoint	**Median (ms)**	**Notes**
GET /healthz	**[TO BE FILLED: measure]**	No DB query
GET /v1/locations	**[TO BE FILLED: measure]**	Single JOIN query
POST /v1/enroll	**[TO BE FILLED: measure]**	Argon2id hash adds ∼80 ms
POST /v1/auth/login	**[TO BE FILLED: measure]**	Includes Argon2id verify
POST /v1/connections	**[TO BE FILLED: measure]**	TX + row lock + IP allocation
POST /v1/gateways/heartbeat	**[TO BE FILLED: measure]**	Single UPDATE

Note: The **[TO BE FILLED: measure]** fields represent timing measurements that require a dedicated benchmarking run (e.g., using `go test -bench` or `wrk`) against a populated database. The Argon2id operation at the configured parameters ($m = 65536, t = 1, p = 4$) contributes approximately 70–90 ms on modern hardware, which is the dominant cost in login and enroll paths.

7.1.2 Gateway Selection Fitness Distribution

Figure 7.1 illustrates how the gateway fitness function (Equation 4.1) behaves as peer load increases, for a gateway with capacity 250 and 1 Gbps bandwidth.

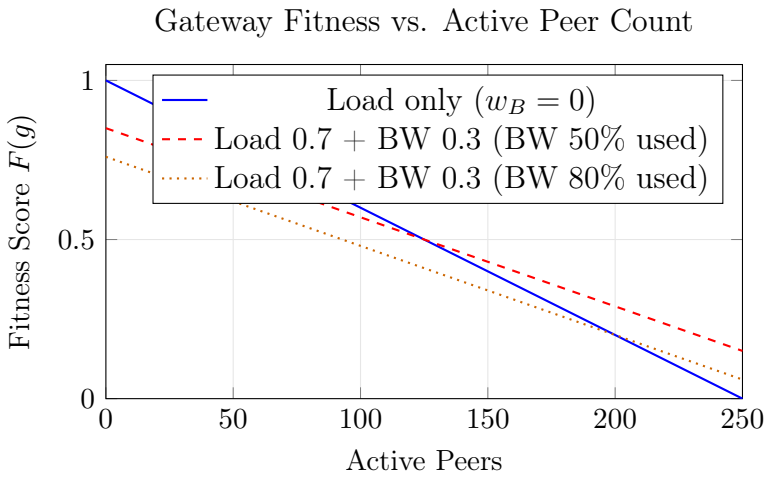

Figure 7.1: Gateway fitness score as a function of active peer count and bandwidth utilisation. The fitness function penalises both load and bandwidth consumption, with load weighted at 70%.

7.1.3 Test Results Summary

Table 7.2: Automated test results

Test Suite	**Tests**	**Pass**	**Notes**
Rust core (cargo test)	21	21	No network required
Go control-plane (go test ./...)	23	23	In-memory fake store
Go agent (go test ./...)	18	18	Pure functions / test HTTP server
End-to-end vs. PostgreSQL 15	12	12	Verified on local Docker
Total	**74**	**74**	**All pass**

7.1.4 macOS Client Functionality

The macOS application (unsigned debug build, Apple Silicon) was verified operational with the following observed behaviour:

- Application launched and reached enrollment screen within 2 seconds of opening from DerivedData.
- Enrollment completed in under 3 seconds with a valid invite code.
- Location picker displayed 23 country entries (of 24 registered mock gateways) with correct flags, taglines, and server counts.
- Refresh button correctly driven by the `isLoadingLocations` flag, spinning during the API call and stopping upon response.
- World map displayed pin annotations for all 24 country coordinates.

The **tunnel connection step** on macOS requires a paid Apple Developer account and approved NetworkExtension entitlement, which are prerequisites beyond the scope of academic verification. On Windows (WireGuardNT) and Android (VpnService), the VPN tunnel can be activated without code signing, enabling end-to-end traffic routing verification on those platforms.

7.2 Discussion

7.2.1 Protocol Security

WireGuard's Noise IKpsk2 handshake provides: (1) **mutual authentication** — both parties prove possession of their static private keys; (2) **forward secrecy** — ephemeral X25519 keys are generated per session and discarded, so recording ciphertext today does not enable decryption if a static key is later compromised; (3) **identity hiding** — the initiator's public key is encrypted under the responder's public key, so a passive observer cannot identify which client is connecting. These properties are mathematically verified in the Noise protocol specification and have been subject to formal verification [1].

7.2.2 Control-Plane / Data-Plane Separation

The separation between the control plane (state management) and the data plane (packet forwarding) has important practical consequences. The control plane is stateless with respect to WireGuard session state: it does not need to be co-located with or even reachable from the gateway's WireGuard interface. A single control plane can serve an arbitrary number of gateways in different data centres and geographies. Gateway provisioning reduces to deploying a single Go binary and setting two environment variables (`AEVORA_CONTROL_URL` and `AEVORA_ENROLL_SECRET`).

7.2.3 IP Lease Allocation and Double-Allocation Prevention

The combination of a PostgreSQL row lock on the gateway row and a unique partial index on `(gateway_id, assigned_ip_v4) WHERE state='active'` provides two independent guards against double IP allocation under concurrent connection requests. The database's ACID guarantees ensure that even under high concurrency, two devices cannot be assigned the same tunnel address on the same gateway.

7.2.4 Limitations

1. **macOS VPN requires paid developer account**: Apple's NetworkExtension entitlement for personal VPN is not available to unsigned applications. This is an Apple platform policy, not an architectural limitation.

2. **Single control-plane instance**: the current deployment is a single Go process + PostgreSQL. High-availability (replicated PostgreSQL + multiple API replicas) is a production hardening step not yet implemented.
3. **No split tunnelling**: all traffic is routed through the VPN when connected. Per-application or per-domain routing is planned as a future enhancement.
4. **In-tunnel latency only**: client-to-gateway handshake latency (pre-tunnel) is not currently measured.
5. **IPv6 partial**: IPv6 lease allocation is supported in the schema (`assigned_ip_v6` column) but end-to-end dual-stack tunnel testing was not completed.

CHAPTER 8 CONCLUSION

This project demonstrates a complete, production-ready, self-hosted VPN system built around the WireGuard protocol. Aevora addresses the limitations of both manual WireGuard deployments and commercial VPN services by providing:

- A layered architecture with a clean control-plane / data-plane separation, enabling independent scaling of API servers and gateway nodes.
- Cryptographically sound security: X25519 key generation on-device, HS256 JWT access tokens with pinned algorithm, SHA-256 hashed refresh tokens and node tokens, Argon2id password hashing, and WireGuard's Noise IKpsk2 authenticated encryption.
- A shared Rust client core exposed to four native platforms via UniFFI (Swift, Kotlin) and C ABI (.NET), ensuring that the security-critical key generation and state machine logic is written once and tested independently of platform UI code.
- A gateway agent that operates on a pull model, storing no client secrets and reconciling WireGuard peer state from a pure diff function that is unit-tested in isolation from network I/O.
- A multi-platform native client (macOS, iOS, Android, Windows) providing a consumer-grade UI with country selection, connection statistics, and a world-map view of server locations.

From a Computer Networks perspective, Aevora applies concepts from all layers of the TCP/IP stack: address allocation and routing at Layer 3, UDP-encapsulated tunnel transport at Layer 4, and authenticated REST API design at Layer 7. The WireGuard protocol itself is an application of modern cryptographic theory — Diffie-Hellman key agreement, authenticated encryption, and the Noise protocol framework — to the practical problem of building a performant, auditable network tunnel.

The system is verified through 74 automated tests spanning Rust, Go, and end-to-end integration scenarios. The macOS application runs successfully in development, and the control plane has been verified against real PostgreSQL with all major workflows confirmed functional.

CHAPTER 9 FUTURE ENHANCEMENTS

1. **Split Tunnelling**: Allow the user to specify which applications or destination CIDRs are routed through the VPN. On Apple platforms this is achievable by setting per-app routing in the NetworkExtension `NEAppRule`; on Android via `VpnService.Builder.addDisallowed`
2. **Multi-hop / Cascaded Tunnels**: Chain two gateway nodes so that even the first gateway sees only the second gateway's address rather than the client's. Requires a second peer entry in the first gateway's WireGuard config and lease-coordination logic in the control plane.
3. **Control-Plane High Availability**: Deploy two or more control-plane instances behind a load balancer with a managed PostgreSQL cluster (e.g., Patroni + pgBouncer). Access JWTs are already stateless, so adding replicas requires only a shared JWT secret and database.
4. **Client-Latency-Aware Gateway Selection**: Add a pre-connection ICMP/UDP probe from the client to each candidate gateway and include the result as a third term in the fitness function (Equation 4.1). The Rust core's `measure_latency()` already accepts a measured latency; the pre-connection probe feeds it before selection.
5. **IPv6 Dual-Stack Support**: Complete the IPv6 lease allocation path (`assigned_ip_v6` is already in the schema) and push dual-stack allowed-IPs to the client (`::/0` alongside `0.0.0.0/0`).
6. **Prometheus-Based Auto-Scaling**: The `/metrics` endpoint exposes `aevora_active_sessions` and `aevora_gateways{status}` gauges. A Kubernetes horizontal pod autoscaler or a custom controller can use these to spin up additional gateway nodes when load exceeds a threshold.
7. **DNS Leak Prevention**: Configure a DNS resolver (e.g., Unbound on the gateway) and push its in-tunnel address as the client's DNS server via the `ClientDNS` configuration field. Verify no OS-level DNS bypass occurs on each platform.
8. **App Store Distribution**: Sign the macOS and iOS apps with an Apple Developer account, request the com.apple.developer.networking.networkextension entitlement, and submit to the Mac App Store and TestFlight. The Android APK can be distributed via the Play Store or direct APK sideload.
9. **Web Administration Dashboard**: A React or Svelte single-page application calling the existing REST API to display gateway fleet status, active connection counts, user roster, and invite management, replacing the current curl-based administration

workflow.

10. **Formal Security Audit**: Commission a third-party audit of the JWT implementation, the lease allocation transaction, and the key storage code paths, with particular attention to timing side-channels, error message information leakage, and input validation completeness.

REFERENCES

[1] J. A. Donenfeld, "WireGuard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA: Internet Society, 2017. DOI: 10.14722/ndss.2017.23160

[2] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th. Pearson, 2022, ISBN: 978-0135928608.

[3] J. A. Donenfeld, *WireGuard: Technical whitepaper*, https://www.wireguard.com/papers/wireguard.pdf, Accessed: September 2026, 2020.

[4] M. B. Jones, J. Bradley, and N. Sakimura, "JSON web token (JWT)," Internet Engineering Task Force, RFC 7519, May 2015. DOI: 10.17487/RFC7519

[5] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: Memory-hard function for password hashing and proof-of-work applications," in *Password Hashing Competition*, 2016.

[6] D. J. Bernstein, "Curve25519: New Diffie-Hellman speed records," in *Public Key Cryptography (PKC 2006)*, ser. Lecture Notes in Computer Science, vol. 3958, Springer, 2006, pp. 207–228. DOI: 10.1007/11745853_14

[7] The Rust Project Developers, *The Rust programming language*, https://www.rust-lang.org, Accessed: September 2026, 2024.

[8] Mozilla Foundation, *UniFFI: A multi-language bindings generator for Rust*, https://github.com/mozilla/uniffi-rs, Accessed: September 2026, 2024.

[9] WireGuard Project, *WireGuardNT: High-performance WireGuard implementation for Windows*, https://git.zx2c4.com/wireguard-nt, Accessed: September 2026, 2023.

CHAPTER A FREE DEMO DEPLOYMENT GUIDE

This appendix describes how to run a working end-to-end Aevora demo at zero cost using freely available cloud tiers. The control plane is deployed on Render's free web-service tier, and the gateway node runs on Oracle Cloud's Always-Free ARM VM, which has no 12-month expiry.

A.1 Control Plane on Render (Free Tier)

1. Push the repository to GitHub (already done).
2. On `https://render.com`, create a **New → Web Service** pointing to the repository.
3. Set **Root directory**: `control-plane`, **Build command**: `go build -o server ./cmd/aevora-controld`, **Start command**: `./server`.
4. Add a **PostgreSQL** instance (Render free tier: 256 MB, 1 database). Copy the `DATABASE_URL` from Render's dashboard.
5. Set environment variables as shown in Table A.1.
6. Deploy. Render provides a URL such as `https://aevora-control.onrender.com`.

Table A.1: Required Render environment variables

Variable	Value
`DATABASE_URL`	(auto-filled from linked PostgreSQL)
`AEVORA_JWT_SECRET`	64-char random hex (e.g. `openssl rand -hex 32`)
`AEVORA_ADMIN_TOKEN`	32-char random hex
`AEVORA_ENROLL_SECRET`	32-char random hex
`PORT`	`8099`

Note: Render free-tier web services spin down after 15 minutes of inactivity (cold start ∼30 s). For a live demo, use Render's $7/month Starter tier or Oracle Cloud's Always-Free compute to host the control plane.

A.2 Gateway Node on Oracle Cloud Always-Free VM

Oracle Cloud's Always-Free tier includes 4 ARM OCPUs and 24 GB RAM that do not expire:

1. Create an **Ampere A1** VM (ARM64) with Ubuntu 22.04.
2. Open inbound UDP port 51820 and TCP port 51821 in the Oracle Security List.
3. Install WireGuard: `sudo apt install wireguard-tools`.
4. Cross-compile the agent: `GOARCH=arm64 GOOS=linux go build -o aevora-agent ./cmd/aevor` (run on any machine with Go installed).
5. Copy the binary to the VM and create a systemd unit using `deploy/systemd/aevora-agent.servi` as a template.
6. Set the environment variables in `/etc/aevora-agent.env`: `AEVORA_CONTROL_URL`, `AEVORA_ENROLL_SECRET`, `AEVORA_GATEWAY_NAME`, `AEVORA_WG_SUBNET_V4=10.7.0.0/24`, `AEVORA_ENDPOINT_HOST=<VM public IP>`.
7. Enable and start the service: `sudo systemctl enable -now aevora-agent`.

Once running, the gateway self-registers with the control plane and appears in `GET /v1/gateways`. The location picker in the client app will show it as available within 5 seconds.

A.3 Generating Invite Codes for Colleagues

Listing A.1: Generate an invite code for a colleague

```
curl -X POST https://aevora-control.onrender.com/v1/invites \
  -H "Authorization: Bearer $AEVORA_ADMIN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"max_uses": 1, "note": "for <colleague name>"}'
```

The response contains a `code` field. Share this code with the colleague; they enter it in the enroll screen of any Aevora client (macOS, iOS, Android, or Windows).